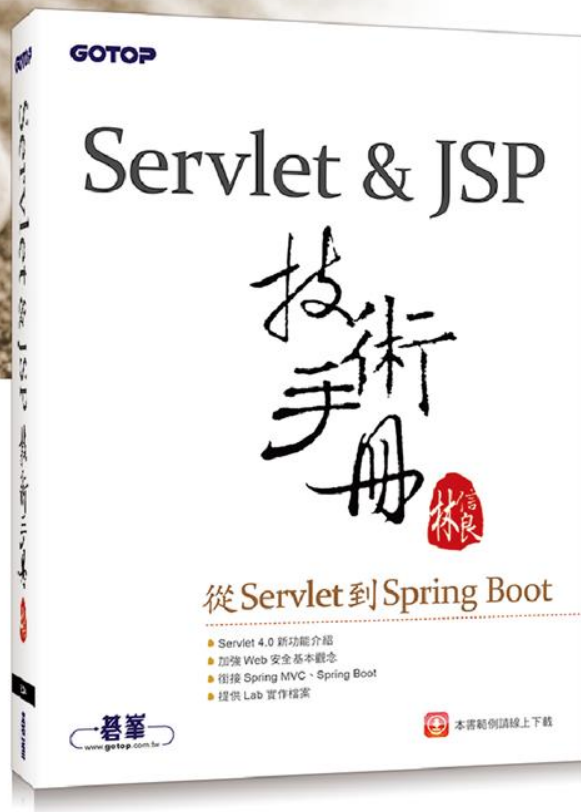




Servlet & JSP

技術手冊

從Servlet到Spring Boot

- Servlet 4.0 新功能介紹
- 加強 Web 安全基本觀念
- 銜接 Spring MVC、Spring Boot
- 提供 Lab 實作檔案

- 整合Spring MVC



學習目標

- 區別程式庫與框架
- 最小套用Spring MVC
- 逐步善用Spring MVC
- 使用Thymeleaf模版

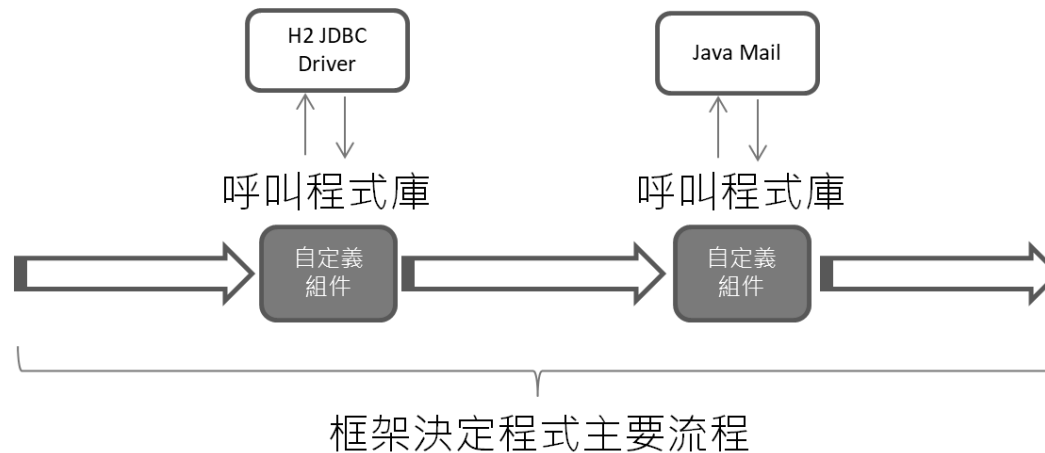
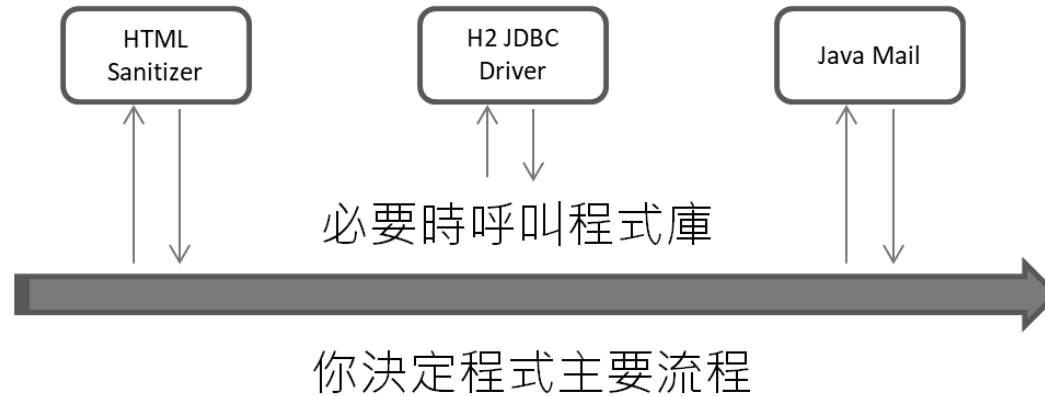
程式庫或框架？

- 開發微網誌的過程中，有使用到幾個程式庫，像是OWASP的HTML Sanitizer、H2 JDBC驅動程式、Java Mail等
- 應用程式主要流程一直在你的控制之內，你決定了何時要處理請求參數、取得模型物件、轉發請求、顯示頁面等各式流程

程式庫或框架？

- 在開始使用框架之後，會發現框架主導了程式運行的流程
- 你必須在框架的規範下定義某些類別，框架會在適當時候調用你實作的程式
- 對應用程式的流程控制權被反轉了，現在是**框架在定義流程**，由框架來呼叫你的程式，而不是由你來呼叫框架。

Inversion of Control



程式庫或框架？

- 會被定位為框架，表示它對程式主要流程擁有更多的控制權
- 框架本身是個半成品
- 要完成整個流程，必須在框架的流程規範下，實現自定義元件

需要使用框架嗎？

- 使用程式庫時，開發者會擁有較高的自由度
- 使用框架時，開發者會受到較大的限制
- 換取而來的益處超越了犧牲掉的流程自由度，才會使得使用框架具有意義

初步套用Spring MVC

- 在gossip專案上按右鍵執行「Configure/Add Gradle Nature」
- 建立build.gradle檔案

```
import org.gradle.plugins.ide.eclipse.model.Facet
apply plugin: 'java'
apply plugin: 'war'
apply plugin: 'eclipse-wtp'

// 設置原始碼版本
sourceCompatibility = 1.8
// 設置 WebApp 根目錄
webAppDirName = 'WebContent'
// 設置 Java 源碼所在目錄
sourceSets.main.java.srcDir 'src'
// 編譯時的原始碼編碼
compileJava.options.encoding = 'UTF-8'

repositories {
    jcenter()
}
```



```
// 設置 Project Facets
eclipse {
    wtp {
        facet {
            facet name: 'jst.web', type: Facet.FacetType.fixed
            facet name: 'wst.jsdt.web', type: Facet.FacetType.fixed
            facet name: 'jst.java', type: Facet.FacetType.fixed
            facet name: 'jst.web', version: '3.1'
            facet name: 'jst.java', version: '1.8'
            facet name: 'wst.jsdt.web', version: '1.0'
        }
    }
}

dependencies {
    // Servlet API
    providedCompile('javax.servlet:javax.servlet-api:4.0.0')
    // JSP API
    providedCompile('javax.servlet.jsp:jsp-api:2.2')

    compile('com.google.guava:guava:21.0')
    compile('com.sun.mail:javax.mail:1.6.0')
    compile('com.h2database:h2:1.4.196')
    compile(
'com.googlecode.owasp-java-html-sanitizer:owasp-java-html-sanitizer:20171016.1'
    )

    compile('org.springframework:spring-context:5.0.3.RELEASE')
    compile('org.springframework:spring-webmvc:5.0.3.RELEASE')
}
```

初始前端控制器

```
package cc.openhome.web;

import org.springframework.web.servlet.support.*;

public class SpringInitializer
    extends AbstractAnnotationConfigDispatcherServletInitializer{

    @Override
    protected Class<?>[] getServletConfigClasses() {
        return new Class<?>[] {WebConfig.class}; ← ❶ Web 層設定
    }

    @Override
    protected Class<?>[] getRootConfigClasses() {
        return new Class<?>[] {RootConfig.class}; ← ❷ 元件層設定
    }

    @Override
    protected String[] getServletMappings() {
        return new String[] {"/"}; ← ❸ 預設 Servlet 路徑
    }
}
```

組態

```
@Configuration
@EnableWebMvc
@ComponentScan("cc.openhome.controller")
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void configureDefaultServletHandling(
        DefaultServletHandlerConfigurer configurer) {
        configurer.enable();
    }
}

@Configuration
public class RootConfig {
}
```

重構控制器

@Controller ← ❶ 這是一個控制器

```
public class MemberController {
```

```
    private String MEMBER_PATH = "/WEB-INF/jsp/member.jsp"; ← ❷ 暫時使用值域取代  
    private String REDIRECT_MEMBER_PATH = "/gossip/member";      Servlet 初始參數
```

@RequestMapping("member") ← ❸ 對 member 的請求會使用此方法

```
public void member(  
    HttpServletRequest request, HttpServletResponse response)  
    throws ServletException, IOException {
```

```
    UserService userService = (UserService)  
        request.getServletContext().getAttribute("userService");
```



❹ 從 request 中取得 ServletContext

```
    List<Message> messages = userService.messages(getUsername(request));
```

```
    request.setAttribute("messages", messages);
```

```
    request.getRequestDispatcher(MEMBER_PATH).forward(request, response);
```

```
}
```

❺ 對 new_message 的 POST 請求
↓ 會使用此方法

```
@RequestMapping(value = "new_message", method = RequestMethod.POST)
```

```
protected void newMessage(  
    HttpServletRequest request, HttpServletResponse response)  
    throws ServletException, IOException {
```

重構控制器

```
@Controller
public class DisplayController {
    private String INDEX_PATH = "/WEB-INF/jsp/index.jsp";
    private String USER_PATH = "/WEB-INF/jsp/user.jsp";

    @RequestMapping("/")
    public void index(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        ...略
    }

    @RequestMapping("user/*")
    public void user(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String username = getUsername(request);
        UserService userService = (UserService)
            request.getServletContext().getAttribute("userService");

        ...略
    }

    private String getUsername(HttpServletRequest request) {
        return request.getRequestURI().replace("/gossip/user/", "");
    }
}
```

在web.xml宣告安全設定

```
<security-constraint>
  <web-resource-collection>
    <web-resource-name>Member</web-resource-name>
    <url-pattern>/del_message</url-pattern>
    <url-pattern>/new_message</url-pattern>
    <url-pattern>/logout</url-pattern>
    <url-pattern>/member</url-pattern>
    <http-method>GET</http-method>
    <http-method>POST</http-method>
  </web-resource-collection>
  <auth-constraint>
    <role-name>member</role-name>
  </auth-constraint>
</security-constraint>

<login-config>
  <auth-method>FORM</auth-method>
  <form-login-config>
    <form-login-page>/</form-login-page>
    <form-error-page>/</form-error-page>
  </form-login-config>
</login-config>

<security-role>
  <role-name>member</role-name>
</security-role>
```

注入服務物件與屬性

@Component

```
public class GmailService implements EmailService {
    private final Properties props = new Properties();

    private final String mailUser;

    private final String mailPassword;

    public GmailService(
        @Value("${mail.user}") String mailUser,
        @Value("${mail.password}") String mailPassword) {

        props.put("mail.smtp.host", "smtp.gmail.com");
        props.put("mail.smtp.auth", "true");
        props.put("mail.smtp.starttls.enable", "true");
        props.put("mail.smtp.port", 587);
        this.mailUser = mailUser;
        this.mailPassword = mailPassword;
    }
    ...略
}
```

```
mail.user=yourname@gmail.com
mail.password=yourpassword
```

```
@Configuration
@PropertySource("classpath:mail.properties") ← ❶ 設定屬性檔案
@ComponentScan("cc.openhome.model") ← ❷ 指定掃描套件
public class RootConfig {
    @Bean
    public DataSource getDataSource() { ← ❸ 管理 DataSource
        try {
            Context initContext = new InitialContext();
            Context envContext = (Context) initContext.lookup("java:/comp/env");
            return (DataSource) envContext.lookup("jdbc/gossip");
        } catch (NamingException e) {
            throw new RuntimeException(e);
        }
    }

    @Bean
    public static PropertySourcesPlaceholderConfigurer ← ❹ 管理屬性資訊
        propertySourcesPlaceholderConfigurer() {
        return new PropertySourcesPlaceholderConfigurer();
    }
}
```



```
path.redirect.member=/gossip/member
path.redirect.index=/gossip
```

```
path.register_success=/WEB-INF/jsp/register_success.jsp
path.register_form=/WEB-INF/jsp/register.jsp
path.verify=/WEB-INF/jsp/verify.jsp
path.forgot=/WEB-INF/jsp/forgot.jsp
path.reset_password_form=/WEB-INF/jsp/reset_password.jsp
path.reset_password_success=/WEB-INF/jsp/reset_success.jsp
```

```
path.index=/WEB-INF/jsp/index.jsp
path.user=/WEB-INF/jsp/user.jsp
path.member=/WEB-INF/jsp/member.jsp
```

```
@Configuration
```

```
@EnableWebMvc
```

```
@PropertySource("classpath:path.properties")
```

```
@ComponentScan("cc.openhome.controller")
```

```
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void configureDefaultServletHandling(
        DefaultServletHandlerConfigurer configurer) {
        configurer.enable();
    }

    @Bean
    public static PropertySourcesPlaceholderConfigurer
        propertySourcesPlaceholderConfigurer() {
        return new PropertySourcesPlaceholderConfigurer();
    }
}
```

```
public class AccountController {
    @Value("${path.redirect.index}")
    private String REDIRECT_INDEX_PATH;

    @Value("${path.register_success}")
    private String REGISTER_SUCCESS_PATH;

    @Value("${path.register_form}")
    private String REGISTER_FORM_PATH;

    @Value("${path.verify}")
    private String VERIFY_PATH;

    @Value("${path.forgot}")
    private String FORGOT_PATH;

    @Value("${path.reset_password_form}")
    private String RESET_PASSWORD_FORM_PATH;

    @Value("${path.reset_password_success}")
    private String RESET_PASSWORD_SUCCESS_PATH;

    ...略

    @Autowired
    private UserService userService;

    @Autowired
    private EmailService emailService;

    ...略
}
```

```
@RequestMapping(value = "register", method = RequestMethod.POST)
public void register(
    HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    ...略

    String path;
    if(errors.isEmpty()) {
        path = REGISTER_SUCCESS_PATH;

        Optional<Account> optionalAcct =
            userService.tryCreateUser(email, username, password);
        if(optionalAcct.isPresent()) {
            emailService.validationLink(optionalAcct.get());
        } else {
            emailService.failedRegistration(username, email);
        }
    } else {
        path = REGISTER_FORM_PATH;
        request.setAttribute("errors", errors);
    }

    request.getRequestDispatcher(path).forward(request, response);
}
...略
```

簡化控制器

```
@Configuration
@EnableWebMvc
@PropertySource("classpath:path.properties")
@ComponentScan("cc.openhome.controller")
public class WebConfig implements WebMvcConfigurer {
    ...略

    @Bean
    public ViewResolver viewResolver() {
        InternalResourceViewResolver resolver =
            new InternalResourceViewResolver();
        resolver.setPrefix("/WEB-INF/jsp/");
        resolver.setSuffix(".jsp");
        resolver.setExposeContextBeansAsAttributes(true);
        return resolver;
    }
}
```

```
path.redirect.member=redirect:/member
path.redirect.index=redirect:/

path.register_success=register_success
path.register_form=register
path.verify=verify
path.forgot=forgot
path.reset_password_form=reset_password
path.reset_password_success=reset_success

path.index=index
path.user=user
path.member=member
```

```
public class AccessController {
```

```
    ...略
```

```
    @RequestMapping(value = "login", method = RequestMethod.POST)
```

```
    public String login(
```

```
        @RequestParam(required=true) String username, ← ❶ 注入請求參數
```

```
        @RequestParam(required=true) String password,
```

```
        HttpServletRequest request) {
```

```
        Optional<String> optionalPasswd =
```

```
            userService.encryptedPassword(username, password);
```

```
        try {
```

```
            request.login(username, optionalPasswd.get());
```

```
            request.getSession().setAttribute("login", username);
```

```
            return REDIRECT_MEMBER_PATH; ← ❷ 傳回路徑字串
```

```
        } catch (NoSuchElementException | ServletException e) {
```

```
            request.setAttribute("errors", Arrays.asList("登入失敗"));
```

```
            List<Message> newest = userService.newestMessages(10);
```

```
            request.setAttribute("newest", newest);
```

```
            return INDEX_PATH;
```

```
        }
```

```
    }
```

```
    @RequestMapping("logout")
```

```
    public String logout(HttpServletRequest request) throws ServletException {
```

```
        request.logout();
```

```
        return REDIRECT_INDEX_PATH;
```

```
    }
```

```
}
```

```
@Controller
public class MemberController {
    ...略

    @RequestMapping("member")
    public String member(
        @SessionAttribute("login") String username, ← ① 注入 HttpSession 属性
        Model model) { ← ② 注入 Model
        List<Message> messages = userService.messages(username);
        model.addAttribute("messages", messages); ← ③ 添加 Model 属性
        return MEMBER_PATH;
    }
}
```

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xmlns:jsp="http://java.sun.com/xml/ns/javaee/jsp"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd" version="4.0">
    ...略
```

```
    <request-character-encoding>UTF-8</request-character-encoding>
</web-app>
```

```
@Controller
@SessionAttributes("token") ← ❶ 指定 HttpSession 屬性名稱
public class AccountController {
    ...略

    @RequestMapping(value = "reset_password", method = RequestMethod.GET)
    public String resetPasswordForm(
        @RequestParam(required=true) String name,
        @RequestParam(required=true) String email,
        @RequestParam(required=true) String token,
        Model model) {

        Optional<Account> optionalAcct =
            userService.accountByNameEmail(name, email);

        if(optionalAcct.isPresent()) {
            Account acct = optionalAcct.get();
            if(acct.getPassword().equals(token)) {
                model.addAttribute("acct", acct);
                model.addAttribute("token", token); ← ❷ 這會是 HttpSession 屬性
                return RESET_PASSWORD_FORM_PATH;
            }
        }
        return REDIRECT_INDEX_PATH;
    }

    ...略
}
```



```
@Controller
public class DisplayController {
    ...略

    @RequestMapping("user/{username}")
    public String user(
        @PathVariable("username") String username,
        Model model) {

        model.addAttribute("username", username);
        if(userService.userExisted(username)) {
            List<Message> messages =
                userService.messages(username);
            model.addAttribute("messages", messages);
        } else {
            model.addAttribute("errors",
                Arrays.asList(String.format("%s 還沒有發表訊息", username)));
        }
        return USER_PATH;
    }
}
```

建立表單物件

- JSR303規範了Java Validation API
- Spring可以整合JSR303
- 需要有個JSR303的實作品，在這邊打算使用Hibernate Validator

```
dependencies {  
    ...略  
  
    compile('org.springframework:spring-context:5.0.3.RELEASE')  
    compile('org.springframework:spring-webmvc:5.0.3.RELEASE')  
  
    compile('org.hibernate:hibernate-validator:5.4.2.Final')  
}
```

建立表單物件

```
public class RegisterForm {
    @Pattern(
        regexp = "^[_a-z0-9-]+(.[_a-z0-9-]+)*@[a-z0-9-]+(.[a-z0-9-]+)*$",
        message = "未填寫郵件或格式不正確"
    )
    private String email;

    @Pattern(regexp = "^\\w{1,16}$", message = "未填寫使用者名稱或格式不正確")
    private String username;

    @Pattern(regexp = "^\\w{8,16}$", message = "請確認密碼符合格式")
    private String password;

    private String password2;

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    ...其他值域相對應的 Getter、Setter，故略...
}
```

建立表單物件

```
public class ResetPasswordForm {  
    private String token;  
    private String name;  
    private String email;  
  
    @Pattern(regexp = "^\\w{8,16}$", message = "請確認密碼符合格式")  
    private String password;  
    private String password2;  
  
    public String getToken() {  
        return token;  
    }  
    public void setToken(String token) {  
        this.token = token;  
    }  
  
    ...其他值域相對應的 Getter、Setter，故略...  
}
```

建立表單物件

```
@Controller
@SessionAttributes("token")
public class AccountController {
    ...略

    @RequestMapping(value = "register", method = RequestMethod.POST)
    public String register(
        @Valid RegisterForm form, ← ❶ 驗證表單物件
        BindingResult bindingResult, ← ❷ 注入驗證結果
        Model model) {
        ❸ 取得 BindingResult 的錯誤訊息
        ↓
        List<String> errors = toList(bindingResult);
        if(!form.getPassword().equals(form.getPassword2())) {
            errors.add("請再度確認密碼");
        }
    }
}
```

建立表單物件

```
private List<String> toList(BindingResult bindingResult) {  
    List<String> errors = new ArrayList<>();  
    if(bindingResult.hasErrors()) { ← ❷ 如果有驗證錯誤訊息的話  
        bindingResult.getFieldErrors().forEach(err -> {  
            errors.add(err.getDefaultMessage());  
        });  
    }  
    return errors;  
}
```



❸ 取得並收集錯誤訊息

關於Thymeleaf模版

- JSP已經過時了？
- JSP確實不是唯一的頁面呈現技術
- 如果使用Spring MVC，在其他模版引擎上，能見度高的選擇之一是Thymeleaf
- 主打的特性之一是自然模版（Natural template）

關於Thymeleaf模版

